✅ TOP OOPS INTERVIEW QUESTIONS & ANSWERS

1. What is OOPS?

Answer:
OOPS (Object-Oriented Programming System) is a programming paradigm based on **objects**, which contain data (attributes) and functions (methods). It helps in modularity, reusability, and organization of large applications.

2. What are the main principles of OOPS?

Answer:
The four key principles are:

1. **Encapsulation** – binding data and methods into a single unit (class).
2. **Abstraction** – showing only essential details and hiding implementation.
3. **Inheritance** – acquiring properties of one class into another.
4. **Polymorphism** – ability to take many forms (method overloading/overriding).

3. What is a Class?

Answer:
A **class** is a blueprint or template for creating objects. It defines attributes (variables) and behaviours (methods).

4. What is an Object?

Answer:
An **object** is an instance of a class. It represents a real-world entity with state and behavior.

5. What is Encapsulation?

Answer:
Encapsulation means **protecting data by restricting direct access** and exposing it through getters/setters.
Example → using private variables.

6. What is Abstraction?

Answer:
Abstraction hides complex implementation and only exposes functionality.
Example → Using a **car accelerator** without knowing engine mechanics.

7. What is Inheritance?

Answer:
Inheritance allows one class (child/derived) to acquire the properties of another (parent/base).
Types:

- Single
- Multilevel
- Hierarchical

- Multiple (in some languages like C++)
- Hybrid

8. What is Polymorphism?

Answer:
Polymorphism means “many forms.”

Two types:

- **Compile-time** → method overloading
- **Run-time** → method overriding

9. What is Method Overloading?

Answer:
Same method name, different parameters within the **same class**.
✓ Happens at compile-time.

10. What is Method Overriding?

Answer:
Same method name and parameters in **parent and child class**.
✓ Happens at runtime.

11. What is Constructor?

Answer:
A special method called automatically when an object is created.
Used to initialize object properties.

12. What are the types of Constructors?

- Default constructor
- Parameterized constructor
- Copy constructor (C++)

13. What is Destructor?

Answer:
A method that destroys an object and frees memory.

14. What is the difference between Abstraction and Encapsulation?

Abstraction	**Encapsulation**
Hides implementation	Protects data
Focus on external view	Focus on internal structure
Achieved using abstract class/interface	Achieved using classes, access modifiers

15. What is the difference between Class and Object?

Class	Object
Template	Instance
Logical entity	Physical entity

16. What is an Interface?

Answer:
An interface contains only abstract methods (and default/static in Java). A class that inherits it must implement its methods.

17. What is Abstract Class?

Answer:
A class containing **abstract methods** (methods without body).
Cannot be instantiated.

18. Difference between Abstract Class and Interface?

Abstract Class	Interface
Can have abstract + concrete methods	Only abstract methods (mostly)
Supports constructors	No constructors
Single inheritance	Multiple inheritance

19. What is Multiple Inheritance?

Answer:
When a class inherits from more than one parent class.
✓ Supported in C++,
✗ Not supported in Java (but done via interfaces).

20. What is "this" keyword?

Answer:
Refers to **current object**.
Used to access current object's variables/methods.

21. What is "super" keyword?

Answer:
Refers to **parent class**.
Used to access parent's methods/variables/constructor.

22. What is a Virtual Function?

Answer:
A function in base class marked as virtual that can be overridden in child class. Enables runtime polymorphism (C++).

23. What is a Pure Virtual Function?

Answer:
A function with no definition: virtual void show() = 0;
Used to create abstract classes.

24. What is Copy Constructor?

Answer:
Constructor that copies values of one object to another. (C++)

25. What is Static Keyword?

Answer:
Used to create class-level members shared across all objects.

26. What is Association?

Answer:
Relationship between two classes. ("uses-a" relationship)

27. Types of Association?

- **Aggregation** (weak relation; independent lifetimes)
- **Composition** (strong relation; dependent lifetimes)

28. Difference between Aggregation and Composition

Aggregation	**Composition**
Weak relation	Strong relation
Can exist independently	Cannot exist independently

29. What is Cohesion?

Answer:
Degree to which class methods are related to each other.
High cohesion → better design.

30. What is Coupling?

Answer:
How much one class depends on another.
Low coupling → better design.

1️⃣ **Scenario: Design a "Bank Account" system using Encapsulation.**

✅ **Requirement**

- balance must be private
- deposit() and withdraw() must control access
- showBalance() for display

✅ **C++ Solution**

```
#include <iostream>
using namespace std;
class BankAccount {
private:
   double balance;
public:
   BankAccount(double amount) {
      balance = amount;
   }

   void deposit(double amount) {
      balance += amount;
   }
   void withdraw(double amount) {
      if(amount <= balance)
         balance -= amount;
      else
         cout << "Insufficient balance\n";
   }
   double getBalance() {
      return balance;
   }
};

int main() {
```

```
    BankAccount b(1000);
    b.deposit(500);
    b.withdraw(300);
    cout << "Balance: " << b.getBalance();
}
```

---

2 Scenario: Company wants different "Employee" types using Inheritance.

Requirement

- Base: Employee
- Derived: FullTimeEmployee, PartTimeEmployee
- Override salary calculation.

C++ Solution

```
#include <iostream>
using namespace std;
class Employee {
public:
    string name;
    Employee(string n): name(n) {}
    virtual double calculateSalary() = 0;
};
class FullTimeEmployee : public Employee {
public:
    double monthlySalary;
    FullTimeEmployee(string n, double s) : Employee(n), monthlySalary(s) {}

    double calculateSalary() {
        return monthlySalary;
    }
};
class PartTimeEmployee : public Employee {
public:
```

```
    double hours, rate;
    PartTimeEmployee(string n, double h, double r) : Employee(n), hours(h), rate(r) {}
    double calculateSalary() {
        return hours * rate;
    }
};

int main() {
    Employee* e1 = new FullTimeEmployee("Rahul", 30000);
    Employee* e2 = new PartTimeEmployee("Amit", 40, 200);
    cout << e1->name << " Salary: " << e1->calculateSalary() << endl;
    cout << e2->name << " Salary: " << e2->calculateSalary() << endl;
}
```

3️⃣ Scenario: Build a "Shape Drawing Tool" using Polymorphism.

Requirement

- Base class: Shape
- Derived: Circle, Rectangle
- Common method area()

C++ Solution

```
#include <iostream>
using namespace std;
class Shape {
public:
    virtual double area() = 0;
};
class Circle : public Shape {
    double r;
public:
    Circle(double radius): r(radius) {}
    double area() {
```

```
        return 3.14 * r * r;
    }
};
class Rectangle : public Shape {
    double l, b;
public:
    Rectangle(double length, double breadth): l(length), b(breadth) {}
    double area() {
        return l * b;
    }
};

int main() {
    Shape* s1 = new Circle(5);
    Shape* s2 = new Rectangle(4, 6);

    cout << "Circle Area: " << s1->area() << endl;
    cout << "Rectangle Area: " << s2->area() << endl;
}
```

4️⃣ **Scenario: Create a "Logger System" using Singleton Pattern.**

Requirement

- Only ONE instance of Logger
- log() method to print messages

C++ Solution

```
#include <iostream>
using namespace std;

class Logger {
private:
    static Logger* instance;
```

```
    Logger() {}

public:
    static Logger* getInstance() {
        if(instance == nullptr)
            instance = new Logger();
        return instance;
    }

    void log(string msg) {
        cout << "[LOG]: " << msg << endl;
    }
};

Logger* Logger::instance = nullptr;

int main() {
    Logger* log1 = Logger::getInstance();
    Logger* log2 = Logger::getInstance();

    log1->log("System Started");
    log2->log("User Logged In");
}
```

5️⃣ Scenario: Library System — Book HAS-A Author (Composition).

Requirement

- Book contains Author
- If Book is destroyed, Author should be destroyed too (Composition).

C++ Solution

```
#include <iostream>
using namespace std;
```

```
class Author {
public:
    string name;
    Author(string n): name(n) {}
};

class Book {
private:
    Author* author;   // Composition
public:
    Book(string authorName) {
        author = new Author(authorName);
    }

    ~Book() { delete author; }

    void show() {
        cout << "Author: " << author->name << endl;
    }
};

int main() {
    Book b("Chetan Bhagat");
    b.show();
}
```

---

6 Scenario: Online Shopping — Product HAS-A Discount Strategy. (Aggregation)

Discount exists independently.

```
#include <iostream>
using namespace std;
```

```
class Discount {
public:
    double rate;
    Discount(double r) : rate(r) {}
};

class Product {
public:
    string name;
    double price;
    Discount* disc;  // Aggregation

    Product(string n, double p, Discount* d)
        : name(n), price(p), disc(d) {}

    double getFinalPrice() {
        return price - (price * disc->rate);
    }
};

int main() {
    Discount d(0.1);
    Product p("Laptop", 60000, &d);
    cout << "Final Price: " << p.getFinalPrice();
}
```

7️⃣ Scenario: Create an Online Payment System using Interfaces (Abstract Class).

Requirement

- Base class: Payment
- Derived: UPI, CardPayment

- Common method: pay()

C++ Solution

```
#include <iostream>
using namespace std;

class Payment {
public:
   virtual void pay(double amount) = 0;
};

class UPI : public Payment {
public:
   void pay(double amount) {
      cout << "Paid " << amount << " using UPI\n";
   }
};

class CardPayment : public Payment {
public:
   void pay(double amount) {
      cout << "Paid " << amount << " using Card\n";
   }
};

int main() {
   Payment* p = new UPI();
   p->pay(500);

   p = new CardPayment();
   p->pay(1200);
}
```

8️⃣ **Scenario: Build a Car System — Car HAS-A Engine, Engine STARTS.**

Demonstrates composition + real behaviour

```
#include <iostream>
using namespace std;

class Engine {
public:
   void start() {
      cout << "Engine Started\n";
   }
};

class Car {
private:
   Engine eng;  // Composition

public:
   void drive() {
      eng.start();
      cout << "Car is moving\n";
   }
};

int main() {
   Car c;
   c.drive();
}
```

9️⃣ **Scenario: Implement Customer Queue using Constructor Overloading.**

```
#include <iostream>
```

```
using namespace std;

class Customer {
public:
    string name;
    int id;

    Customer() {
        name = "Unknown";
        id = 0;
    }

    Customer(string n) {
        name = n;
        id = 0;
    }

    Customer(string n, int i) {
        name = n;
        id = i;
    }
};

int main() {
    Customer c1;
    Customer c2("Ravi");
    Customer c3("Amit", 101);

    cout << c3.name << " ID: " << c3.id;
}
```

🔟 **Scenario: Create a Student Result System using Operator Overloading.****

Add marks of two students

```
#include <iostream>
using namespace std;

class Student {
public:
   int marks;

   Student(int m) : marks(m) {}

   Student operator+(Student &s) {
      return Student(marks + s.marks);
   }
};

int main() {
   Student s1(40), s2(50);
   Student s3 = s1 + s2;

   cout << "Total Marks: " << s3.marks;
}
```

1️⃣ Design a Ride-Sharing System (Ola/Uber) using OOPS.

Requirements

Driver and Customer classes

Matching algorithm

Polymorphic Payment

Encapsulation of location

Dynamic dispatch

C++ Solution

```
#include <iostream>
using namespace std;

class Location {
private:
    double lat, lon;
public:
    Location(double la, double lo) : lat(la), lon(lo) {}
};

class Rider {
public:
    string name;
    Location loc;

    Rider(string n, Location l) : name(n), loc(l) {}
};

class Driver {
public:
    string name;
    double rating;
    Location loc;

    Driver(string n, double r, Location l) : name(n), rating(r), loc(l) {}
```

```
};

class Payment {
public:
   virtual void pay(double amount) = 0;
};

class UpiPayment : public Payment {
public:
   void pay(double amount) {
      cout << "Paid " << amount << " using UPI\n";
   }
};

class CardPayment : public Payment {
public:
   void pay(double amount) {
      cout << "Paid " << amount << " using Card\n";
   }
};

class RideManager {
public:
   Driver matchDriver() {
      // Hard-coded driver
      return Driver("Amit", 4.9, Location(10, 20));
   }

   void startRide(Rider r, Payment* p) {
      Driver d = matchDriver();
      cout << "Driver " << d.name << " matched for " << r.name << '\n';
```

```
        p->pay(350);  // fixed fare
    }
};

int main() {
    Rider r("Rahul", Location(10, 10));
    Payment *p = new UpiPayment();

    RideManager rm;
    rm.startRide(r, p);
}
```

2️⃣ Design a Hospital Management System (Inheritance + Polymorphism).

Requirements:

Doctor, Nurse, and Patient classes

schedule(), treat() must behave differently

C++ Solution

```
#include <iostream>
using namespace std;

class Person {
public:
    string name;
    Person(string n) : name(n) {}

    virtual void duty() = 0;
};
```

```
class Doctor : public Person {
public:
   Doctor(string n) : Person(n) {}
   void duty() {
      cout << name << " treats patients\n";
   }
};

class Nurse : public Person {
public:
   Nurse(string n) : Person(n) {}
   void duty() {
      cout << name << " assists doctor\n";
   }
};

class Patient : public Person {
public:
   Patient(string n) : Person(n) {}
   void duty() {
      cout << name << " waits for treatment\n";
   }
};

int main() {
   Person* p1 = new Doctor("Dr. Sharma");
   Person* p2 = new Nurse("Nisha");
   Person* p3 = new Patient("Ravi");

   p1->duty();
   p2->duty();
```

```
    p3->duty();
}
```

3️⃣ Design a Food-Delivery System (Swiggy/Zomato).

Requirements:

Restaurant HAS-A Menu

User selects item

Delivery boy assigned

Strong Composition

C++ Solution

```
#include <iostream>
#include <vector>
using namespace std;

class FoodItem {
public:
    string name;
    double price;
    FoodItem(string n, double p) : name(n), price(p) {}
};

class Restaurant {
private:
    vector<FoodItem> menu;
public:
    Restaurant() {
```

```
        menu.push_back(FoodItem("Burger", 150));
        menu.push_back(FoodItem("Pizza", 300));
    }
    FoodItem getItem(int idx) { return menu[idx]; }
};

class DeliveryBoy {
public:
    string name;
    DeliveryBoy(string n) : name(n) {}
};

class Order {
private:
    FoodItem item;
    DeliveryBoy boy;
public:
    Order(FoodItem f, DeliveryBoy d) : item(f), boy(d) {}

    void deliver() {
        cout << "Delivering " << item.name << " by " << boy.name << endl;
    }
};

int main() {
    Restaurant r;
    DeliveryBoy d("Suresh");

    Order o(r.getItem(1), d);
    o.deliver();
}
```

4️⃣ Design an ATM System (Abstraction + Polymorphism).

Requirements:

Card authentication

Cash withdrawal

Multiple account types

C++ Solution

```
#include <iostream>
using namespace std;

class Account {
public:
    double balance;

    Account(double b) : balance(b) {}
    virtual void withdraw(double amt) = 0;
};

class Savings : public Account {
public:
    Savings(double b) : Account(b) {}
    void withdraw(double amt) {
        if (amt <= balance) balance -= amt;
        cout << "Savings New Balance: " << balance << endl;
    }
};
```

```
class Current : public Account {
public:
   Current(double b) : Account(b) {}
   void withdraw(double amt) {
      balance -= amt; // overdraft allowed
      cout << "Current New Balance: " << balance << endl;
   }
};

class ATM {
public:
   void process(Account* acc, double amt) {
      acc->withdraw(amt);
   }
};

int main() {
   Account* a = new Savings(5000);
   ATM atm;
   atm.process(a, 1500);
}
```

5️⃣ Design a Smart-Home Automation System.

Requirements:

Light, Fan, AC behave differently

TurnOn() & TurnOff() polymorphic

Controller class to manage all devices

C++ Solution

```
#include <iostream>
#include <vector>
using namespace std;

class Device {
public:
    virtual void turnOn() = 0;
    virtual void turnOff() = 0;
};

class Light : public Device {
public:
    void turnOn() { cout << "Light ON\n"; }
    void turnOff() { cout << "Light OFF\n"; }
};

class Fan : public Device {
public:
    void turnOn() { cout << "Fan ON\n"; }
    void turnOff() { cout << "Fan OFF\n"; }
};

class Controller {
private:
    vector<Device*> devices;

public:
    void addDevice(Device* d) { devices.push_back(d); }

    void activateAll() {
```

```
        for (auto d : devices) d->turnOn();
    }
};
```

6️⃣ Design a Stock Trading App (Observer Pattern + OOPS).

Requirements:

Stock price changes

All subscribed users notified

Observer pattern

C++ Solution

```
#include <iostream>
#include <vector>
using namespace std;

class Investor {
public:
    virtual void update(double price) = 0;
};

class Stock {
private:
    double price;
    vector<Investor*> subs;
public:
    void addSubscriber(Investor* i) { subs.push_back(i); }

    void setPrice(double p) {
```

```
        price = p;
        for (auto &s : subs) s->update(price);
    }
};

class MobileApp : public Investor {
public:
    void update(double price) {
        cout << "Mobile App Alert: New Stock Price = " << price << endl;
    }
};
```

7️⃣ Design a Movie Streaming System (Netflix-like).

OOPS requirements:

Movie class

Account class

Playback depends on subscription type → polymorphism

C++ Example

```
#include <iostream>
using namespace std;

class Subscription {
public:
    virtual void playMovie() = 0;
};

class Basic : public Subscription {
```

```
public:
   void playMovie() { cout << "Playing in 480p\n"; }
};

class Premium : public Subscription {
public:
   void playMovie() { cout << "Playing in 1080p + HDR\n"; }
};

class User {
public:
   string name;
   Subscription* s;

   User(string n, Subscription* sub) : name(n), s(sub) {}

   void watch() {
      cout << name << " watching movie...\n";
      s->playMovie();
   }
};
```

8️⃣ Design a Warehouse Inventory System (Operator Overloading).

Requirement:

Add inventory of two warehouses using operator +

C++ Code

```
#include <iostream>
using namespace std;
```

```
class Inventory {
public:
   int items;
   Inventory(int i) : items(i) {}

   Inventory operator+(Inventory &inv) {
      return Inventory(items + inv.items);
   }
};
```

9️⃣ Airline Ticket Booking System (Exception Handling + OOPS).

Requirements:

Book ticket

Throw exception if seats unavailable

C++ Code

```
#include <iostream>
using namespace std;

class Flight {
   int seats;
public:
   Flight(int s) : seats(s) {}

   void book(int count) {
      if(count > seats)
         throw runtime_error("No seats available!");
      seats -= count;
      cout << "Booked " << count << " seats\n";
```

```
    }
};
```

🔟 Hotel Room Reservation System (Aggregation + Composition).

Hotel HAS-A Room

Customer chooses room

Rooms independent → aggregation

C++ Code

```
#include <iostream>
using namespace std;

class Room {
public:
    int number;
    bool available;
    Room(int n): number(n), available(true) {}
};

class Hotel {
public:
    Room* room;
    Hotel(Room* r) : room(r) {}

    void book() {
        if(room->available) {
            room->available = false;
            cout << "Room " << room->number << " booked!\n";
```

```
        }
        else cout << "Not available\n";
    }
};
```

Parking Lot System (C++ Sample)

cpp

```
enum class VehicleType { Car, Bike, Truck };

class Vehicle {
protected:
   std::string licensePlate;
   VehicleType type;
public:
   Vehicle(std::string plate, VehicleType t) : licensePlate(plate), type(t) {}
   virtual VehicleType getType() const { return type; }
   virtual ~Vehicle() {}
};

class ParkingSpot {
   bool occupied;
   VehicleType spotType;
   Vehicle* vehicle;
public:
   ParkingSpot(VehicleType t) : spotType(t), occupied(false), vehicle(nullptr) {}
   bool canFit(Vehicle* v) { return !occupied && v->getType() == spotType; }
   void park(Vehicle* v) { if (canFit(v)) { vehicle = v; occupied = true; } }
   void leave() { occupied = false; vehicle = nullptr; }
};

class ParkingFloor {
   std::vector<ParkingSpot> spots;
public:
   ParkingFloor(int n, VehicleType t) { for (int i=0; i<n; ++i) spots.emplace_back(t); }
   bool parkVehicle(Vehicle* v) { for (auto& s : spots) if (s.canFit(v)) { s.park(v); return true; } return
false; }
```

```
};

class ParkingLot {
    static ParkingLot* instance;
    std::vector<ParkingFloor> floors;
    ParkingLot() {}
public:
    static ParkingLot* getInstance() { if (!instance) instance = new ParkingLot; return instance; }
    void addFloor(const ParkingFloor& f) { floors.push_back(f); }
};

ParkingLot* ParkingLot::instance = nullptr;
```

- Concepts: Singleton, Polymorphism, Composition.

ATM System (C++ Sample)

cpp

```
class Card {
    std::string cardNumber;
    double balance;
public:
    Card(std::string num, double bal) : cardNumber(num), balance(bal) {}
    double getBalance() const { return balance; }
    void debit(double amt) { if (balance >= amt) balance -= amt; }
    void credit(double amt) { balance += amt; }
};

class BankServer {
public:
    void processWithdrawal(Card& c, double amt) { c.debit(amt); }
    void processDeposit(Card& c, double amt) { c.credit(amt); }
```

```
};

class ATM {
    BankServer server;
public:
    void withdraw(Card& card, double amount) { server.processWithdrawal(card, amount); }
    void deposit(Card& card, double amount) { server.processDeposit(card, amount); }
};
```

- Concepts: Encapsulation, Error Handling.

Elevator System (C++ Sample)

cpp

```
enum class Direction { UP, DOWN, IDLE };

class Elevator {
    int currentFloor;
    Direction direction;
    std::set<int> requests;
public:
    Elevator(int startFloor = 0) : currentFloor(startFloor), direction(Direction::IDLE) {}
    void moveUp() { ++currentFloor; direction = Direction::UP; }
    void moveDown() { --currentFloor; direction = Direction::DOWN; }
    void addRequest(int floor) { requests.insert(floor); }
    void stop() { direction = Direction::IDLE; requests.erase(currentFloor); }
};

class ElevatorController {
    std::vector<Elevator> elevators;
public:
    ElevatorController(int n) { for (int i=0; i<n; ++i) elevators.emplace_back(0); }
```

```
    void assignRequest(int floor) { elevators[0].addRequest(floor); /* naive assignment for simplicity */
}

};
```

- Concepts: Scheduling, Enums, Controller.